Structures (Unit 4)

Arrays allow to define type of variables that can hold several data items of the same kind. Similarly **structure** is another user defined data type available in C that allows to combine data items of different kinds.

Structures are used to represent a record. Suppose you want to keep track of your books in a library. You might want to track the following attributes about each book −

- Title
- Author
- Subject
- Book ID

Defining a Structure

To define a structure, you must use the **struct** statement. The struct statement defines a new data type, with more than one member. The format of the struct statement is as follows −

```
struct [structure tag] {

   member definition;
   member definition;
   ...
   member definition;
} [one or more structure variables];
```

The **structure tag** is optional and each member definition is a normal variable definition, such as int i; or float f; or any other valid variable definition. At the end of the structure's definition, before the final semicolon, you can specify one or more structure variables but it is optional. Here is the way you would declare the Book structure −

```
struct Books {
   char  title[50];
   char author[50];
   char subject[100];
   int   book_id;
} book;
```

Accessing Structure Members

To access any member of a structure, we use the **member access operator (.)**. The member access operator is coded as a period between the structure variable name and the structure member that we wish to access. You would use the

keyword **struct** to define variables of structure type. The following example shows how to use a structure in a program −

```
#include <stdio.h>
#include <string.h>

struct Books {
   char  title[50];
   char author[50];
   char subject[100];
   int   book_id;
};

int main( ) {

   struct Books Book1;        /* Declare Book1 of type Book */
   struct Books Book2;        /* Declare Book2 of type Book */

   /* book 1 specification */
   strcpy( Book1.title, "C Programming");
   strcpy( Book1.author, "Nuha Ali");
   strcpy( Book1.subject, "C Programming Tutorial");
   Book1.book_id = 6495407;

   /* book 2 specification */
   strcpy( Book2.title, "Telecom Billing");
   strcpy( Book2.author, "Zara Ali");
   strcpy( Book2.subject, "Telecom Billing Tutorial");
   Book2.book_id = 6495700;

   /* print Book1 info */
   printf( "Book 1 title : %s\n", Book1.title);
   printf( "Book 1 author : %s\n", Book1.author);
   printf( "Book 1 subject : %s\n", Book1.subject);
   printf( "Book 1 book_id : %d\n", Book1.book_id);

   /* print Book2 info */
   printf( "Book 2 title : %s\n", Book2.title);
   printf( "Book 2 author : %s\n", Book2.author);
   printf( "Book 2 subject : %s\n", Book2.subject);
   printf( "Book 2 book_id : %d\n", Book2.book_id);

   return 0;
}
```

When the above code is compiled and executed, it produces the following result −

```
Book 1 title : C Programming
Book 1 author : Nuha Ali
Book 1 subject : C Programming Tutorial
```

```
Book 1 book_id : 6495407
Book 2 title : Telecom Billing
Book 2 author : Zara Ali
Book 2 subject : Telecom Billing Tutorial
Book 2 book_id : 6495700
```

Structures as Function Arguments

You can pass a structure as a function argument in the same way as you pass any other variable or pointer.

```c
#include <stdio.h>
#include <string.h>

struct Books {
   char  title[50];
   char author[50];
   char subject[100];
   int   book_id;
};

/* function declaration */
void printBook( struct Books book );

int main( ) {

   struct Books Book1;        /* Declare Book1 of type Book */
   struct Books Book2;        /* Declare Book2 of type Book */

   /* book 1 specification */
   strcpy( Book1.title, "C Programming");
   strcpy( Book1.author, "Nuha Ali");
   strcpy( Book1.subject, "C Programming Tutorial");
   Book1.book_id = 6495407;

   /* book 2 specification */
   strcpy( Book2.title, "Telecom Billing");
   strcpy( Book2.author, "Zara Ali");
   strcpy( Book2.subject, "Telecom Billing Tutorial");
   Book2.book_id = 6495700;

   /* print Book1 info */
   printBook( Book1 );

   /* Print Book2 info */
   printBook( Book2 );
```

```
   return 0;
}

void printBook( struct Books book ) {

   printf( "Book title : %s\n", book.title);
   printf( "Book author : %s\n", book.author);
   printf( "Book subject : %s\n", book.subject);
   printf( "Book book_id : %d\n", book.book_id);
}
```

When the above code is compiled and executed, it produces the following result −

```
Book title : C Programming
Book author : Nuha Ali
Book subject : C Programming Tutorial
Book book_id : 6495407
Book title : Telecom Billing
Book author : Zara Ali
Book subject : Telecom Billing Tutorial
Book book_id : 6495700
```

Pointers to Structures

You can define pointers to structures in the same way as you define pointer to any other variable −

```
struct Books *struct_pointer;
```

Now, you can store the address of a structure variable in the above defined pointer variable. To find the address of a structure variable, place the '&'; operator before the structure's name as follows −

```
struct_pointer = &Book1;
```

To access the members of a structure using a pointer to that structure, you must use the → operator as follows −

```
struct_pointer->title;
```

Let us re-write the above example using structure pointer.

```
#include <stdio.h>
#include <string.h>

struct Books {
   char  title[50];
```

```c
   char author[50];
   char subject[100];
   int   book_id;
};

/* function declaration */
void printBook( struct Books *book );
int main( ) {

   struct Books Book1;        /* Declare Book1 of type Book */
   struct Books Book2;        /* Declare Book2 of type Book */

   /* book 1 specification */
   strcpy( Book1.title, "C Programming");
   strcpy( Book1.author, "Nuha Ali");
   strcpy( Book1.subject, "C Programming Tutorial");
   Book1.book_id = 6495407;

   /* book 2 specification */
   strcpy( Book2.title, "Telecom Billing");
   strcpy( Book2.author, "Zara Ali");
   strcpy( Book2.subject, "Telecom Billing Tutorial");
   Book2.book_id = 6495700;

   /* print Book1 info by passing address of Book1 */
   printBook( &Book1 );

   /* print Book2 info by passing address of Book2 */
   printBook( &Book2 );

   return 0;
}

void printBook( struct Books *book ) {

   printf( "Book title : %s\n", book->title);
   printf( "Book author : %s\n", book->author);
   printf( "Book subject : %s\n", book->subject);
   printf( "Book book_id : %d\n", book->book_id);
}
```

When the above code is compiled and executed, it produces the following result −

```
Book title : C Programming
Book author : Nuha Ali
Book subject : C Programming Tutorial
Book book_id : 6495407
Book title : Telecom Billing
Book author : Zara Ali
Book subject : Telecom Billing Tutorial
```

Book book_id : 6495700

Operations on struct variables in C

In C, the only operation that can be applied to *struct* variables is assignment. Any other operation (e.g. equality check) is not allowed on *struct* variables.
For example, program 1 works without any error and program 2 fails in compilation.

Example:

```
#include <stdio.h>

struct Point {
  int x;
  int y;
};

int main()
{
  struct Point p1 = {10, 20};
  struct Point p2 = p1; // works: contents of p1 are copied to p2
  printf(" p2.x = %d, p2.y = %d", p2.x, p2.y);
  getchar();
  return 0;
}
```

Example:

```
#include <stdio.h>

struct Point {
  int x;
  int y;
};

int main()
{
  struct Point p1 = {10, 20};
  struct Point p2 = p1; // works: contents of p1 are copied to p2
  if (p1 == p2)  // compiler error: cannot do equality check for
                 // whole structures
  {
    printf("p1 and p2 are same ");
  }
  getchar();
  return 0;
```

```
}
```

Nested Structure in C

C provides us the feature of nesting one structure within another structure by using which, complex data types are created. For example, we may need to store the address of an entity employee in a structure. The attribute address may also have the subparts as street number, city, state, and pin code. Hence, to store the address of the employee, we need to store the address of the employee into a separate structure and nest the structure address into the structure employee. Consider the following program.

```
#include<stdio.h>
struct address
{
    char city[20];
    int pin;
    char phone[14];
};
struct employee
{
    char name[20];
    struct address add;
};
void main ()
{
    struct employee emp;
    printf("Enter employee information?\n");
    scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.ad
d.phone);
    printf("Printing the employee information... \n");
    printf("name: %s\nCity: %s\nPincode: %d\nPhone: %s",emp.name,em
p.add.city,emp.add.pin,emp.add.phone);
}
```

Output

```
Enter employee information?
```

```
Arun

Delhi

110001

1234567890

Printing the employee information....

name: Arun

City: Delhi

Pincode: 110001

Phone: 1234567890
```

The structure can be nested in the following ways.

1. By separate structure
2. By Embedded structure

1) Separate structure

Here, we create two structures, but the dependent structure should be used inside the main structure as a member. Consider the following example.

```
struct Date
{
   int dd;
   int mm;
   int yyyy;
};
struct Employee
```

```
{
   int id;
   char name[20];
   struct Date doj;
}emp1;
```

As you can see, doj (date of joining) is the variable of type Date. Here doj is used as a member in Employee structure. In this way, we can use Date structure in many structures.

2) Embedded structure

The embedded structure enables us to declare the structure inside the structure. Hence, it requires less line of codes but it can not be used in multiple data structures. Consider the following example.

```
struct Employee
{
   int id;
   char name[20];
   struct Date
    {
      int dd;
      int mm;
      int yyyy;
    }doj;
}emp1;
```

Accessing Nested Structure

We can access the member of the nested structure by Outer_Structure.Nested_Structure.member as given below:

```
e1.doj.dd
e1.doj.mm
e1.doj.yyyy
```

C Nested Structure example

Let's see a simple example of the nested structure in C language.

```
#include <stdio.h>
#include <string.h>
struct Employee
```

```
{
   int id;
   char name[20];
   struct Date
    {
      int dd;
      int mm;
      int yyyy;
    }doj;
}e1;
int main( )
{
   //storing employee information
   e1.id=101;
   strcpy(e1.name, "Sonoo Jaiswal");//copying string into char arra
y
   e1.doj.dd=10;
   e1.doj.mm=11;
   e1.doj.yyyy=2014;

   //printing first employee information
   printf( "employee id : %d\n", e1.id);
   printf( "employee name : %s\n", e1.name);
   printf( "employee date of joining (dd/mm/yyyy) : %d/%d/%d\n", e1
.doj.dd,e1.doj.mm,e1.doj.yyyy);
   return 0;
}
```

Output:

```
employee id : 101
employee name : Sonoo Jaiswal
employee date of joining (dd/mm/yyyy) : 10/11/2014
```

Passing structure to function

Just like other variables, a structure can also be passed to a function. We may pass the structure members into the function or pass the structure variable at once. Consider the following example to pass the structure variable employee to a function display() which is used to display the details of an employee.

```
#include<stdio.h>
struct address
{
    char city[20];
    int pin;
    char phone[14];
```

```
};
struct employee
{
    char name[20];
    struct address add;
};
void display(struct employee);
void main ()
{
    struct employee emp;
    printf("Enter employee information?\n");
    scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.ad
d.phone);
    display(emp);
}
void display(struct employee emp)
{
  printf("Printing the details ...\n");
  printf("%s %s %d %s",emp.name,emp.add.city,emp.add.pin,emp.add.ph
one);
}
```

Union

Like structure, **Union in c language** is *a user-defined data type* that is used to store the different type of elements.

At once, only one member of the union can occupy the memory. In other words, we can say that the size of the union in any instance is equal to the size of its largest element.

Defining union

The **union** keyword is used to define the union. Let's see the syntax to define union in c.

```
union union_name
{
    data_type member1;
    data_type member2;
    .
    .
    data_type memeberN;
};
```

```
Let's see the example to define union for an employee in c.
union employee
{   int id;
    char name[50];
    float salary;
};
```

C Union example

Let's see a simple example of union in C language.

```
#include <stdio.h>
#include <string.h>
union employee
{   int id;
    char name[50];
}e1; //declaring e1 variable for union
int main( )
{
   //store first employee information
   e1.id=101;
   strcpy(e1.name, "Sonoo Jaiswal");//copying string into char arra
y
   //printing first employee information
   printf( "employee 1 id : %d\n", e1.id);
   printf( "employee 1 name : %s\n", e1.name);
   return 0;
}
```

Output:

```
employee 1 id : 1869508435
employee 1 name : Sonoo Jaiswal
```

As you can see, id gets garbage value because name has large memory size. So only name will have actual value.

Difference between Structure and Union in C

structures in C

A structure is a user-defined data type available in C that allows to combining data items of different kinds. Structures are used to represent a record.

Defining a structure: To define a structure, you must use the **struct** statement. The struct statement defines a new data type, with more than or equal to one member. The format of the struct statement is as follows:

```
struct [structure name]
{
    member definition;
    member definition;
    ...
    member definition;
};
```

union

A union is a special data type available in C that allows storing different data types in the same memory location. You can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple purposes.

Defining a Union: To define a union, you must use the union statement in the same way as you did while defining a structure. The union statement defines a new data type with more than one member for your program. The format of the union statement is as follows:

```
union [union name]
{
   member definition;
   member definition;
   ...
   member definition;
};
```

Similarities between Structure and Union

1. Both are user-defined data types used to store data of different types as a single unit.

2. Their members can be objects of any type, including other structures and unions or arrays. A member can also consist of a bit field.
3. Both structures and unions support only assignment = and sizeof operators. The two structures or unions in the assignment must have the same members and member types.
4. A structure or a union can be passed by value to functions and returned by value by functions. The argument must have the same type as the function parameter. A structure or union is passed by value just like a scalar variable as a corresponding parameter.
5. '.' operator is used for accessing members.

Differences

	STRUCTURE	UNION
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is **greater than or equal to the sum of sizes of its members.**	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of **union is equal to the size of largest member.**
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

```
// C program to illustrate differences
// between structure and Union
#include <stdio.h>
#include <string.h>

// declaring structure
struct struct_example
{
    int integer;
    float decimal;
    char name[20];
};

// declaraing union
```

```
union union_example
{
    int integer;
    float decimal;
    char name[20];
};

void main()
{
    // creating variable for structure
    // and initializing values difference
    // six
    struct struct_example s={18,38,"geeksforgeeks"};

    // creating variable for union
    // and initializing values
    union union_example u={18,38,"geeksforgeeks"};


    printf("structure data:\n integer: %d\n"
                "decimal: %.2f\n name: %s\n",
                s.integer, s.decimal, s.name);
    printf("\nunion data:\n integeer: %d\n"
                 "decimal: %.2f\n name: %s\n",
                u.integer, u.decimal, u.name);


    // difference two and three
    printf("\nsizeof structure : %d\n", sizeof(s));
    printf("sizeof union : %d\n", sizeof(u));

    // difference five
    printf("\n Accessing all members at a time:");
    s.integer = 183;
    s.decimal = 90;
    strcpy(s.name, "geeksforgeeks");

    printf("structure data:\n integer: %d\n "
                "decimal: %.2f\n name: %s\n",
            s.integer, s.decimal, s.name);

    u.integer = 183;
    u.decimal = 90;
    strcpy(u.name, "geeksforgeeks");

    printf("\nunion data:\n integeer: %d\n "
                "decimal: %.2f\n name: %s\n",
            u.integer, u.decimal, u.name);
```

```
    printf("\n Accessing one member at time:");

    printf("\nstructure data:");
    s.integer  =  240;
    printf("\ninteger: %d", s.integer);

    s.decimal  =  120;
    printf("\ndecimal: %f", s.decimal);

    strcpy(s.name, "C programming");
    printf("\nname: %s\n", s.name);

    printf("\n union data:");
    u.integer = 240;
    printf("\ninteger: %d", u.integer);

    u.decimal  =  120;
    printf("\ndecimal: %f", u.decimal);

    strcpy(u.name, "C programming");
    printf("\nname: %s\n", u.name);

    //difference four
    printf("\nAltering a member value:\n");
    s.integer = 1218;
    printf("structure data:\n integer: %d\n "
                " decimal: %.2f\n name: %s\n",
                s.integer, s.decimal, s.name);

    u.integer = 1218;
    printf("union data:\n integer: %d\n"
           " decimal: %.2f\n name: %s\n",
            u.integer, u.decimal, u.name);
}
```

Output:

```
structure data:
 integer: 18
 decimal: 38.00
 name: geeksforgeeks

union data:
 integeer: 18
 decimal: 0.00
 name: ?

sizeof structure: 28
sizeof union: 20
```

```
 Accessing all members at a time: structure data:
 integer: 183
 decimal: 90.00
 name: geeksforgeeks

union data:
 integeer: 1801807207
 decimal: 277322871721159510000000000.00
 name: geeksforgeeks

 Accessing one member at a time:
structure data:
integer: 240
decimal: 120.000000
name: C programming

 union data:
integer: 240
decimal: 120.000000
name: C programming

Altering a member value:
structure data:
 integer: 1218
 decimal: 120.00
 name: C programming
union data:
 integer: 1218
 decimal: 0.00
 name: ?
```

In my opinion, structure is better because as memory is shared in union ambiguity is more.